

Reviewer Pack — Spectral Gap Determines Tseitin Resolution Length

Entry 715ab68a1aef · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 11:20:22 UTC

Spectral Gap Determines Tseitin Resolution Length

Entry ID: 715ab68a1aef **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 11:20:22 UTC

1. Conjecture

Field A (mathematical branch): Spectral Graph Theory **Field B** (complexity object): Resolution Proof Length

Statement:

For any connected graph G with $n \geq 3$, the resolution length of its Tseitin formula is $\geq 2^{\{\Omega(\mu)\}}$, where μ is the algebraic connectivity (smallest non-zero Laplacian eigenvalue) of G . For non-expanders ($\mu \leq 1/n$), this bound is polynomial in n .

Rationale (proposer's reasoning):

Algebraic connectivity captures graph expansion properties critical for Tseitin formula hardness. Spectral methods provide a concrete, poly-time computable invariant linking graph structure to resolution complexity, avoiding algebraization barriers.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1d5d0ef2e4af6994

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 10 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - algebraic connectivity Tseitin formula resolution proof - spectral gap proof complexity Tseitin formula - exponential lower bound algebraic connectivity resolution length Tseitin

Top relevant hits considered: - [<http://arxiv.org/abs/math/0701895v1>] An algebraic proof of Deligne's regularity criterion for integrable connections - [<http://arxiv.org/abs/2103.09609v1>] Characterizing Tseitin-formulas with short regular resolution refutations - [<http://arxiv.org/abs/2209.05839v3>] On bounded depth proofs for Tseitin formulas on the grid; revisited - [<http://arxiv.org/abs/2403.19550v2>] Spectral gap for surfaces of infinite volume with negative curvature - [<http://arxiv.org/abs/2508.16353v2>] On the asymptotic behavior of the spectral gap for discrete Schrödinger operators - [<http://arxiv.org/abs/1009.0906v3>] Spectral asymptotics for Robin problems with a discontinuous coefficient - [<http://arxiv.org/abs/0906.0693v3>] An improved lower bound on the counterfeit coins problem - [<http://arxiv.org/abs/1007.1875v2>] Lower Bounds for Quantum Oblivious Transfer

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.7s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_random_graph(n):
    edges = set()
    for i in range(n):
        for j in range(i + 1, n):
            if random.random() < 0.5:
                edges.add((i, j))
    return edges

def laplacian_matrix(G, n):
    L = [[0] * n for _ in range(n)]
    for i in range(n):
        degree = sum(1 for j in range(n) if (i, j) in G or (j, i) in G)
        L[i][i] = -degree
        for j in range(i + 1, n):
            if (i, j) in G or (j, i) in G:
                L[i][j] = 1
                L[j][i] = 1
    return L

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i
        for j in range(i + 1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(i + 1, n):
```

```

        factor = Fraction(A[j][i], A[i][i])
        for k in range(n):
            A[j][k] -= factor * A[i][k]
    return A

def determinant(A):
    n = len(A)
    det = Fraction(1)
    for i in range(n):
        for j in range(i + 1, n):
            factor = Fraction(A[j][i], A[i][i])
            for k in range(i + 1, n):
                A[j][k] -= factor * A[i][k]
        det *= A[i][i]
    return det

def algebraic_connectivity(G, n):
    L = laplacian_matrix(G, n)
    eigenvalues = []
    for i in range(n):
        identity = [[Fraction(1) if j == k else Fraction(0) for j in range(n)] for k in range(n)]
        A = [row[:] for row in L]
        A[i][i] -= 1
        det_A = determinant(A)
        eigenvalues.append(det_A / n)
    return min(eigenvalue for eigenvalue in eigenvalues if eigenvalue > 0)

def Tseitin_formula(G, n):
    literals = [f"x{i}" for i in range(n)]
    clauses = []
    for i in range(n):
        clause = [literals[i]]
        for j in range(i + 1, n):
            if (i, j) not in G and (j, i) not in G:
                clause.append(f"~{literals[j]}")
        clauses.append(clause)
    return literals, clauses

def resolution_length(clauses):
    queue = [clause[:] for clause in clauses]
    while queue:
        clause_i = queue.pop(0)
        if len(clause_i) == 1:
            return len(queue) + 1
        for clause_j in queue:
            common_lits = set(lit for lit in clause_i if f"~{lit}" in clause_j)
            if common_lits:
                new_clause = [lit for lit in clause_i + clause_j if lit not in common_lits and f"~{lit}" not in clause_j]
                queue.append(new_clause)
    return float('inf')

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(3, 40)
    G = generate_random_graph(n)
    μ = algebraic_connectivity(G, n)

```

```

if  $\mu \leq 1/n$ :
    c = 1
else:
    c = 1/10
Tseitin_lits, Tseitin_clauses = Tseitin_formula(G, n)
length = resolution_length(Tseitin_clauses)
conjecture_holds = length >= 2**(c *  $\mu$ )
counterexample = "" if conjecture_holds else f" $\mu$ ={ $\mu$ }, length={length}"
return {
    "metric_name": "resolution_length",
    "metric_value": length,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 1 for i in range(5, 8)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...{result}...}}")
        results.append(result)

    mean_length = sum(result["metric_value"] for result in results) / len(results)
    std_length = math.sqrt(sum((result["metric_value"] - mean_length)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_length} std={std_length} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results) and support_fraction >= 0.8:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        counterexample = next(result["counterexample"] for result in results if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_84b6cba1.py", line 123, in <module>
    result = run_trial(seed)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_84b6cba1.py", line 101, in run_trial
     $\mu$  = algebraic_connectivity(G, n)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_84b6cba1.py", line 71, in algebraic_connectivity
    return min(eigenvalue for eigenvalue in eigenvalues if eigenvalue > 0)
    ~~~~~^~~~~~

```

ValueError: min() iterable argument is empty

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed during algebraic connectivity calculation, preventing data collection. The error suggests potential issues with graph generation or eigenvalue computation for connected graphs. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	55621
2	preregistration	ollama_remote	qwen3:8b	0	0	27528
3	novelty	ollama_remote	qwen3:8b	0	0	22272
4	novelty	ollama_remote	qwen3:8b	0	0	40626
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40303
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21672
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26696
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15044
9	judge	ollama_remote	qwen3:8b	0	0	26492

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 276253 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/715ab68a1aef.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/715ab68a1aef.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/715ab68a1aef.tar.gz` (if generated)